Swayam Singh

singhswayam008@gmail.com +91 9116277756 GitHub LinkedIn Google Scholar

Experience

Quansight

Software Engineer II · Full-time 07/2026 – Present

- Re-architected vLLM's multi-process serving stack into a single free-threaded (no-GIL) process, cutting inter-engine coordination overhead ∼13.3x and P99 inter-token latency from ∼540 ms to ∼112 ms.
- Contribute upstream to CPython, NumPy, PyTorch, and CuPy, ensuring compatibility, correctness, and performance across the scientific Python and AI stack.

Open Source Intern – Numerical Types & Precision (NumPy OSS) · Internship 07/2024 – 09/2024

- Built multi-platform distribution and testing infrastructure handling compiler toolchains, SIMD availability, FMA support, and architecture-specific behavior.
- Led development of NumPy-QuadDType (cross-platform 128-bit float dtype); implemented casting, ufunc dispatch, scalar types, memory model, and serialization.

Neocambrian AI

Member of Technical Staff – Reinforcement Learning

- Design and build RL environments for cybersecurity, AI research, robot policy control, and vision reasoning on sandboxed, containerized execution with scalable parallel-rollout infrastructure.
- Engineered verifiable, hacking-resistant reward pipelines: programmatic exploit checks for attacks on simulated hardware (e.g., DMA), cryptographic schemes, sandbox escapes, and planted cache-eviction bugs; automated grading of end-to-end ML experiments; and grounded answer verification.
- Building a Physical AI RSI (recursive self-improvement) framework for robot policy learning, where policies generate their own simulated experience to drive successive rounds of policy and reward refinement.

Microsoft Research 07/2024 – 06/2026

Research Fellow

- Ran multi-node distributed training on Kubernetes across NVIDIA B200 clusters: pod scheduling, GPU resource allocation, and distributed-throughput optimization for large-scale code-model training.
- Built the world's first end-to-end autoformalization and verification pipeline for Rust: a GRPO-based RL recipe lifted verified programs from 1% to 47% over SFT; our 4B Verus-LM beats Kimi-K2.5 (1T). Also trained an LLM on par with Gemini and GPT-4o via online/offline RL.
- Engineered a fused multi-GPU inference megakernel for DeepSeek-V2 (236B) on B200, merging all compute and TP/EP communication into one kernel via multimem PTX multicast, block-scaled FP8, and warp specialization.

NumPy (numpy-quaddtype) 12/2024 – Present

Maintainer

- Authored and maintain a cross-platform, true IEEE-compliant 128-bit quadruple-precision floating-point data type for NumPy, along with corresponding efficient and performant BLAS routines.
- Review and merge core PRs affecting dtype architecture, numerical semantics, and cross-platform behavior; focus on dtype infrastructure, numerical correctness, and long-term ABI stability.
- Work with contributors across architectures on portability, performance, and ABI concerns.

dataX.ai 05/2022 – 10/2022

Machine Learning Engineer Intern

- Deployed vision/language models via NVIDIA Triton Inference Server (ONNX conversion pipeline), cutting VM load by 12%.
- Wrote custom CUDA kernels for 3D medical-scan processing, achieving a 2x speedup in segmentation over existing GPU solutions.

Projects

- **Free-Threaded vLLM Serving (Python no-GIL, CUDA)**: Re-architected vLLM's multi-process inference server into a single free-threaded (no-GIL) process, replacing ZMQ/msgpack and SHM/pickle IPC between the API layer, scheduler, and per-GPU workers with in-process reference passing, and removing the single-uvloop API-server bottleneck via per-frontend threads. Cut inter-engine coordination overhead ∼13.3x (94.7 ms to 7.1 ms wall-clock union) and P99 inter-token latency from ∼540 ms to ∼112 ms; best configuration reached 61.96 req/s vs 59.33 stock. Made CUDA graphs compatible in the decode path and added a thread-local tokenizer.
- **Bare-Bones Inference (CUDA, C++)**: A minimal inference stack with engine-grade features such as batching and scheduling, but serving a single fused megakernel instead of a conventional multi-kernel pipeline. Built DeepSeek-V2 (236B)'s block-scaled FP8-quantized, multi-GPU megakernel for NVIDIA B200 (sm_100) GPUs that fuses all computation and TP/EP communication into one kernel, handling cross-rank exchange via `multimem` PTX

multicast writes (directly writing into peer ranks' memory at once). Achieves 13.11 ms/token decode latency, beating vLLM's optimized serving latency at B=1 by 2x and eager-mode by >10x.
- **Blackwell Flash-Attention Kernels (CUDA, PTX)**: Prototyped flash-attention kernels for NVIDIA Blackwell (B200, sm_100) using tcgen05 2SM MMA and warp specialization, with mbarrier-based producer/consumer synchronization and a persistent-CTA design; profiled against roofline limits using CuTe layouts.
- **NVFP4 MoE Training System (CUDA, C++)**: Built a Mixture-of-Experts training system from scratch targeting NVIDIA B200 with NVFP4 (4-bit) numerics, focused on low-precision training stability and multi-GPU throughput.
- **Numpy-QuadDType (C, C++, Python)**: A cross-platform 128-bit (quadruple-precision) floating-point dtype for NumPy with 100k+ downloads. A portable alternative to long double with full casting, ufunc dispatch, scalar types, and serialization that behaves consistently across compilers and architectures.
- **QBLAS (C, C++)**: A high-performance BLAS library for IEEE-754 binary128 (quadruple) precision, implementing optimized linear-algebra kernels that bring quad-precision numerics to workloads unsupported by standard double-precision libraries.
- **cpp-verify (C++, LLVM, SMT)**: Extends C++ with first-class formal-verification constructs (pre/post-conditions and invariants), lowering specifications through an LLVM-based pipeline and discharging the resulting proof obligations to SMT solvers.

Publications

- **NextCoder: Robust Adaptation of Code LMs to Diverse Code Edits** (ICML 2025; ICLR 2025 DL4Code). **Swayam Singh**, Tushar Aggarwal, et al. Synthetic-data pipeline + SeleKT adaptation; 32B matches Gemini and beats GPT-4o on harder edit tasks, purely via SFT.
- **Narrow Transformer: StarCoder-Based Java-LM For Desktop** (arXiv:2407.03941, 2024). Kamalkumar Rathinasamy, ..., **Swayam Singh**, et al. A compact, Java-specialized code LM for desktop hardware.
- **OctoPack: Instruction Tuning Code Large Language Models** (ICLR 2024 Spotlight; NeurIPS 2023 Instruction Workshop). Niklas Muennighoff, ..., **Swayam Singh**, et al. Instruction tuning of code models via natural-language Git commits (CommitPack).
- **StarCoder: May the Source Be With You!** (TMLR 2023). Raymond Li, ..., **Swayam Singh**, et al. (BigCode). A 15.5B open code LLM trained on 1T tokens of permissively licensed code.

Skills Summary

- **Systems, GPU & Serving**: CUDA kernels and megakernels, multi-GPU / multi-node deployment, tensor/expert parallelism, cross-rank collectives (multimem / NVLink), CUDA graphs, FP8 quantization, inference serving (vLLM internals), NVIDIA Triton Inference Server, SIMD/FMA, performance optimization
- **Infra & Distributed**: Kubernetes (multi-node GPU workloads, scheduling, resource allocation), distributed training, cross-platform C-extension packaging & CI
- **Languages**: C, C++, CUDA, CuTeDSL, Python (incl. free-threaded / no-GIL)
- **ML & Training**: LLM post-training, reinforcement learning (online/offline RL), supervised fine-tuning, code-generation models
- **Tools & Frameworks**: PyTorch, CuPy, NumPy (C-API internals), LLVM, SMT solvers
- **Open Source**: Core NumPy maintenance, cross-platform C-extension packaging, code review

Honors and Awards

- **2024** – **Kaggle Competition Expert**: Bronze medal (top 7%) in UBC-OCEAN; top 3% in the *30 Days of ML* challenge.
- **2024** – Invited to **Google Research Week** (keynote by Jeff Dean).
- **2024** – **OctoPack** accepted as a **Spotlight (top 5%)** at ICLR 2024.
- **2023** – Selected for the **Amazon ML Summer School 2023**.
- **2023** – **Clothes Virtual Try-On** crossed 500+ GitHub stars.

Invited Talks

- **MAMBA: Zero to Hero**, invited talk on State Space Models at Cohere for AI.
- **Provably-Correct Code and Efficient Sparse Training of LLMs**, internal research talk at Microsoft Research.
- **Foundations of Machine Learning**, a GDG On-Campus session on the ML landscape, tooling ecosystem, and emerging directions.
- **From Deep Learning to Large Language Models**, a talk on the foundations of modern AI: deep learning, generative models, and LLMs.